

Pre-Merge Audit — EXZIBO Vercel Serverless Functions

Pre-Merge Audit — EXZIBO Vercel Serverless Functions

1. Executive Conclusion

PART A — Verified Current Facts

PART C — Merger Evaluation

PART D — Recommended Architecture

Serverless Function Consolidation Analysis

EXZIBO — Restaurant Management Platform

Vercel API Routes & Serverless Functions — Full Codebase Audit

Analysis-only report · No code modified · July 14, 2026

Pre-Merge Audit — EXZIBO Vercel

Serverless Functions

Type: Analysis only — no code, configuration, or database changes were made while producing this report. **Repository state**

audited: `api/*.js`, `api/_lib/*`, `vercel.json`, `vite.config.js`, `server.js`, `src/lib/db.js`, `src/db/*`, `drizzle/migrations/*`, Better Auth config, Upstash usage, Cloudflare realtime + R2 usage, all server-side `process.env` references.

1. Executive Conclusion

The 12 files in `api/*.js` are already the physical unit of deployment on Vercel — there is **no low-level over-fragmentation problem**. Each file uses an internal `?action=` router, so the “12 functions” already behave like 40+ logical endpoints multiplexed onto 12 physical ones.

Of the three proposed mergers:

- **B (`menu.js` + `content.js`) is SAFE WITH CONDITIONS.** Both files already share the same `?action=` dispatch convention, have zero action-name collisions, and use compatible middleware (CORS, Upstash rate limiting). A thin-router merge is realistic.
- **A (`settings.js` + `team.js`) is DO NOT MERGE as a flat merge.** The two handlers use incompatible dispatch conventions — `settings.js` unconditionally rejects any request without `?action=`, while `team.js`’s primary GET route (`/api/team-members/:restaurantId` → `/api/team?restaurantId=:restaurantId`) sends **no action parameter at all**. Merging them as-is would 400 every team-member list request.
- **C (`orders.js` + `bookings.js`) is DO NOT MERGE.** Both handlers use **bare, action-less GET and POST as their primary routes** (`?restaurantId=X` for list, plain POST body for

create). A merged handler has no way to tell an order-list request from a booking-list request, or an order-creation POST from a booking-creation POST, without changing every `vercel.json` rewrite that targets them today — which violates the “preserve public URLs” constraint.

The more valuable consolidation opportunity is **not** merging the 12 Vercel functions — it is eliminating the **three-way duplication** of business logic across `vite.config.js` (dev), `server.js` (Replit runtime), and `api/*.js` (Vercel production), which currently reimplement the same menu/orders/bookings/team/content logic independently and have already drifted (see §6).

The single most serious finding is **not architectural but a security gap**: `api/restaurants.js`’s `permanentDelete` and `softDelete` actions, and most of `team.js`, `menu.js`, `content.js`, `notifications.js`, and `bookings.js`, have **no server-side authorization check at all** — restaurant-data isolation currently depends entirely on the caller knowing a restaurant UUID. This is a pre-existing condition, not something introduced or worsened by the proposed mergers, and per the governing rules is reported here without a remediation plan.

Go/No-Go: See §17. Short answer — proceed with **Merger B only**, under the conditions in §8; do **not** proceed with Mergers A or C under the current URL scheme.

PART A — Verified Current Facts

2. Exact Physical Vercel Function Count

12 physical Vercel functions (one per file in `api/*.js`, each exporting a single default handler — this is what Vercel deploys as one serverless function per file):

#	File	Physical function
1	<code>api/auth.js</code>	□
2	<code>api/auth-check.js</code>	□
3	<code>api/restaurants.js</code>	□
4	<code>api/settings.js</code>	□
5	<code>api/team.js</code>	□

#	File	Physical function
6	api/menu.js	☐
7	api/content.js	☐
8	api/media.js	☐
9	api/orders.js	☐
10	api/bookings.js	☐
11	api/notifications.js	☐
12	api/system.js	☐

Not physical Vercel functions (classification, per the audit's required distinctions):

- **vercel.json rewrites** — 38 rewrite rules that remap legacy/ REST-style public URLs (e.g. `/api/menu/items/:restaurantId`) onto the 12 real functions via query strings (e.g. `/api/menu?action=getItems&restaurantId=...`). These are routing config, not code, and do not count toward the function total.
- **Shared utility files** — `api/_lib/cors.js`, `api/_lib/authz.js` are imported helpers with no exported HTTP handler; not deployed as functions.
- **Vite development middleware** — 8 dev-only plugins in `vite.config.js` (`preview-auth`, `menu-api`, `about-api`, and others) that exist only for `vite dev`; never deployed to Vercel.
- **Express routes** — ~30 routes in `server.js`, used only by the Replit `npm run dev` workflow (`node server.js` is not what Vercel runs); never deployed to Vercel.
- **Cloudflare Workers / Durable Objects** — `exzibo-realtime/src/index.ts`, a separate Cloudflare Worker with a `MyDurableObject` Durable Object class, deployed independently to Cloudflare, not Vercel.

3. Current Function Inventory

Legend for **Security boundary**: `public` (no check), `authenticated` (valid session required for at least one action), `restaurant-scoped` (data implicitly scoped by an `id` param but not verified against session), `role-protected` (role checked), `superadmin-only`.

3.1 `api/auth.js`

- **Public paths / rewrite:** `/api/auth/:path*` → `/api/auth?_path=:path*`
- **Methods/actions:** All HTTP methods, delegated wholesale to Better Auth's `toNodeHandler`
- **Frontend callers:** `src/lib/auth-client.js` (Better Auth client SDK), all login/logout/session UI
- **Request/response:** Better Auth's own wire protocol (OAuth redirects, session JSON)
- **DB tables:** Better Auth's `user`, `session`, `account`, `verification` (snake_case-mapped)
- **Shared helpers/services:** `src/lib/auth.server.js` (`auth`, `ensureAuthSchema`), Google OAuth
- **Env vars:** `BETTER_AUTH_BASE_URL`, `BETTER_AUTH_URL`, `BETTER_AUTH_SECRET`, `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`, `DATABASE_URL`, `BETTER_AUTH_TRUSTED_ORIGINS`
- **Security boundary:** `public` (this is the authentication surface)
- **Upstash/realtime/R2:** none
- **Runtime deps:** `better-auth`, `pg`
- **Side effects:** schema bootstrap (`ensureAuthSchema()`) runs on every cold start
- **Duplicate impl:** none — single source of truth in all three runtimes (dev/Replit/Vercel all import the same `auth.server.js`)

3.2 `api/auth-check.js`

- **Public paths:** `/api/auth-check` (no rewrite — hit directly with `?type=`)
- **Methods/actions:** `GET ?type=superadmin`, `GET ?type=member&restaurantId=...`
- **Frontend callers:** RBAC routing layer (`RestaurantDashboard.jsx` and superadmin gate) — used to decide which dashboard UI to render
- **Response:** `{ allowed, role, isSuperadmin, email }`
- **DB tables:** `restaurant_members` (via `checkRestaurantAccess`)
- **Shared helpers:** `checkSuperadmin`, `checkRestaurantAccess` from `api/_lib/authz.js`

- **Env vars:** `SUPERADMIN_ALLOWED_EMAILS` , `DATABASE_URL`
- **Security boundary:** `role-protected` / `superadmin-only`
depending on `type` — this file **is** the authorization boundary for the rest of the app
- **Side effects:** none destructive
- **Duplicate impl:** `server.js` has an equivalent `_isPrivateAdminPath` guard using the same `auth.server.js` ;
`vite.config.js` has no equivalent global guard (see §6)

3.3 `api/restaurants.js`

- **Public paths / rewrites:** `/api/neon/restaurants` , `/api/neon/restaurant/create` , `/api/neon/restaurant/by-slug/:slug` , `/api/neon/restaurant/by-uid/:uid` , `/api/neon/restaurant/:id` , `/api/restaurant/:id` , `/api/restaurant/update-profile`
- **Methods/actions:** GET : `list` , `listDeleted` , `bySlug` , `byId` , `checkSlug` , `myIds` , `neonRestaurant` ; POST : `create` , `update` , `updateProfile` , `softDelete` , `permanentDelete` ; PATCH : `neonRestaurant`
- **Frontend callers:** `src/lib/db.js` — `getRestaurants` , `getDeletedRestaurants` , `getRestaurantBySlug` , `getRestaurantById` , `checkLinkNameTakenInDB` , `updateRestaurant` , `updateRestaurantProfile` , `deleteRestaurant` / `permanentDeleteRestaurant` , `createRestaurant`
- **DB tables:** `restaurants` (read/write), and on `permanentDelete` : cascading manual `DELETE` on `orders` , `bookings` , `menu_items` , `menu_categories` , `restaurant_members` , `restaurant_about` , `restaurant_settings` , `restaurants` (`api/restaurants.js:168-175`)
- **Shared helpers:** `getSessionEmail` (`api/_lib/authz.js`), `src/db/neon-restaurants.js` , `src/db/pg-sql.js`
- **Env vars:** `DATABASE_URL` , `VITE_DISABLE_AUTH` , `DISABLE_AUTH`
- **Security boundary — mixed, action by action:**
 - `myIds` — `authenticated` (401 if no session; bypassed entirely when `DISABLE_AUTH=true` , `api/restaurants.js:100-107`)
 - `create` — `authenticated` best-effort (session used only to fill `owner_id` if present; a request with no session still succeeds — `api/restaurants.js:131-137`)

- `list`, `listDeleted`, `bySlug`, `byId`, `checkSlug`, `neonRestaurant` (GET), `update`, `updateProfile`, `softDelete` — **public**, **no server-side check**
- `permanentDelete` — **public**, **no server-side check**, and **destructive** (cascades across 7 tables) — **critical finding**, see §5
- **Upstash/realtime:** none used in this file
- **Side effects/destructive:** `permanentDelete` is an irreversible multi-table cascade with no auth, no confirmation, no audit log entry
- **Duplicate impl:** `vite.config.js` (`system` middleware block, 575-898) and `server.js` (268-461) reimplement restaurant CRUD independently

3.4 `api/settings.js`

- **Public paths / rewrite:** `/api/settings` (direct), `/api/neon/restaurant-settings/shadow-upsert` → `/api/settings?action=setRestaurantSettings`
- **Methods/actions:** GET : `getGlobal`, `getUserSettings`, `getRestaurantSettings`; POST : `setGlobal`, `saveUserSettings`, `setRestaurantSettings`
- **Every action requires `?action=`**; missing action → 400 unconditionally (`api/settings.js:26`)
- **Frontend callers:** `src/lib/db.js` — `getUserSettings`, `saveUserSettings`
- **DB tables:** `global_settings`, `user_settings` (via `src/db/neon-globals.js`), `restaurant_settings` (direct SQL + `src/db/neon-restaurant-settings.js`)
- **Shared helpers:** `getSessionEmail`, `neon()` from `src/db/pg-sql.js`
- **Env vars:** `DATABASE_URL`, `VITE_DISABLE_AUTH`, `DISABLE_AUTH`
- **Security boundary:**
 - `getUserSettings`, `saveUserSettings` — **authenticated** (401 without session; bypassed under `DISABLE_AUTH`)
 - `getGlobal`, `setGlobal`, `getRestaurantSettings`, `setRestaurantSettings` — **public**, no check (`setGlobal` is an unauthenticated global write)

- **Side effects:** none destructive
- **Duplicate impl:** `vite.config.js` `system` block and `server.js:1035-1054` reimplement `setRestaurantSettings` shadow-upsert

3.5 `api/team.js`

- **Public paths / rewrites:** `/api/team-members/:restaurantId` → `/api/team?restaurantId=:restaurantId` (no `action` param), `/api/team-members/shadow-upsert` → `?action=shadowUpsert`, `/api/team-members/shadow-delete` → `?action=shadowDelete`
- **Methods/actions:** `GET` (no action — plain `restaurantId` list, `api/team.js:24-29`); `POST`: `create`, `update`, `shadowUpsert` (all alias the same upsert path, `api/team.js:34-39`), `delete`, `shadowDelete` (alias the same delete path, `api/team.js:42-47`)
- **Frontend callers:** `src/lib/db.js` — `getTeamMembers`, `createTeamMember` / `updateTeamMember`, `deleteTeamMember`
- **DB tables:** `restaurant_members`
- **Shared helpers:** `src/db/neon-restaurant-members.js`
- **Env vars:** none referenced directly in this file
- **Security boundary:** `public` — **no server-side session/role check anywhere in this file**. Anyone who knows a `restaurantId` can list, add, update, or delete staff members.
- **Side effects/destructive:** `deleteNeonRestaurantMember(id)` — no auth, no soft-delete
- **Duplicate impl:** `vite.config.js` `menu-api` block and `server.js:849-887` reimplement team CRUD independently; `create` / `update` / `shadowUpsert` are already **duplicate aliases within this one file**

3.6 `api/menu.js`

- **Public paths / rewrites:** 9 rewrites (`/api/menu/categories/*`, `/api/menu/items/*`) all mapped to `?action=`
- **Methods/actions:** `GET`: `getCategories`, `getItems`, `getPublishedItems`; `POST`: `createItem`, `upsertItems`, `updateItem`, `deleteItem`, `upsertCategory`, `deleteCategory`
- **Frontend callers:** `src/lib/db.js` — `getMenuCategories`, `upsertMenuCategory`, `deleteMenuCategory`, `getMenuItems`,

- `getPublishedMenuItems`, `insertMenuItem`, `upsertMenuItems`, `updateMenuItem` / `toggleMenuItemPublish`, `deleteMenuItem`
- **DB tables:** `menu_categories`, `menu_items` (via `src/db/neon-menu-categories.js`, `src/db/neon-menu-items.js`)
- **Shared helpers/services:** Upstash (`rateLimit`, `acquireLock`, `releaseLock`, `getClientIp`, `send429` from `src/lib/upstash.server.js`)
- **Env vars:** Upstash creds (indirect, via `upstash.server.js`)
- **Security boundary:** `public` — **no server-side session/role check anywhere in this file.** Protection is rate-limit-only.
- **Upstash keys:** `rl:menu-create:ip:{ip}` (30/60s), `rl:menu-upsert:ip:{ip}` (10/60s), `rl:menu-update:ip:{ip}` (60/60s), `rl:menu-delete:ip:{ip}` (20/60s), `rl:category-upsert:ip:{ip}` (30/60s), `rl:category-delete:ip:{ip}` (20/60s); locks `lock:menu-item:{id}` (5s), `lock:menu-category:{id}` (5s)
- **Side effects/destructive:** `deleteItem`, `deleteCategory` are hard deletes guarded only by a 5s advisory lock, not auth
- **Duplicate impl:** `vite.config.js` `menu-api` block (`100-435`) and `server.js` (`470-665`) reimplement all menu CRUD independently, **without** the Upstash rate-limit/lock guards present in `api/menu.js` — a real dev/prod behavioral difference (see §6)

3.7 `api/content.js`

- **Public paths / rewrites:** `/api/about/:restaurantId` → `getAbout`, `/api/about/save` → `saveAbout`, `/api/restaurant/update-social` → `updateSocial`
- **Methods/actions:** GET : `getAbout` ; POST : `saveAbout`, `updateSocial`
- **Frontend callers:** `src/lib/db.js` (about/story text + social links save flows), `RestaurantWebsite.jsx` (public read of about content)
- **DB tables:** `restaurant_about` (via `src/db/neon-restaurant-about.js`), `restaurants.social_links` (via `patchNeonRestaurant`)
- **Shared helpers/services:** Upstash rate limiting
- **Security boundary:** `public` — no session/role check

- **Upstash keys:** `rl:about-save:ip:{ip}` (10/60s), `rl:social-update:ip:{ip}` (20/60s)
- **Side effects:** none destructive
- **Duplicate impl:** `vite.config.js` `about-api` block (437-564) and `server.js` (904-1033)

3.8 `api/media.js`

- **Public paths / rewrites:** `/api/menu/upload-image`, `/api/about/upload-image`, `/api/restaurant/upload-logo`, `/api/restaurant/upload-carousel` — all mapped to distinct `?action=`
- **Methods/actions:** `POST` only: `uploadMenuImage`, `uploadAboutImage`, `uploadLogoImage`, `uploadCarouselImage`
- **Frontend callers:** `src/lib/db.js` — `uploadDataUrlToStorage`, `uploadCarouselImageViaApi`, `uploadLogoViaApi`, `uploadMenuImage`
- **Request contract:** base64 `dataUrl` + `restaurantId` (+ `slot` for about images) in JSON body, `bodyParser.sizeLimit: '10mb'` (`api/media.js:12`)
- **Response:** `{ url, imageKey }`
- **Shared helpers/services:** `r2Upload` (`src/lib/r2.js`), Upstash rate limiting
- **Env vars (via `src/lib/r2.js`):** `R2_ACCOUNT_ID`, `R2_ACCESS_KEY_ID`, `R2_SECRET_ACCESS_KEY`, `R2_BUCKET_NAME`, `R2_PUBLIC_BASE_URL` / `R2_PUBLIC_URL`
- **R2 operations:** `PUT` only (no delete/get in this file) — key patterns `restaurants/{id}/menu-items/{ts}.webp`, `restaurants/{id}/about/image-{slot+1}-{ts}.webp`, `restaurants/{id}/logo/{ts}.webp`, `restaurants/{id}/carousel/{ts}.webp`
- **Security boundary:** `public` — no session/role check; protected only by `rl:upload:ip:{ip}` (15/60s)
- **Side effects:** writes to external R2 storage; old images are never deleted (storage grows unbounded on re-upload)
- **Runtime-specific:** larger body size limit than any other function (10 MB vs. default)
- **Duplicate impl:** `vite.config.js` `about-api` / `menu-api` blocks and `server.js` reimplement R2 upload calls independently, each importing `src/lib/r2.js` directly rather than sharing one upload service

3.9 `api/orders.js`

- **Public paths / rewrites:** `/api/orders/:restaurantId` → GET list, `/api/orders/update-status`, `/api/orders/auto-cleanup`
- **Methods/actions:** GET (no action, `restaurantId` list); POST **no action** = create order; POST `?action=updateStatus`; POST `?action=autoCleanup`
- **Frontend callers:** `src/lib/db.js` — `getOrders`, `createOrder`, `updateOrderStatus`; cron/internal caller for `autoCleanup` (also self-triggered on server boot per workflow logs: `[auto-cleanup] Neon [ ] deleted 0 completed + 0 rejected`)
- **DB tables:** `orders` (via `src/db/neon-orders.js`)
- **Shared helpers/services:** Upstash rate-limit/lock, `publishOrderRealtimeEvent` (`src/lib/realtime-publisher.js`)
- **Realtime events:** `ORDER_CREATED` (on create), `ORDER_STATUS_CHANGED` (on status update) — POSTed to `${REALTIME_URL}/publish/order-event` with `Authorization: Bearer REALTIME_PUBLISH_SECRET`
- **Env vars:** Upstash creds, `REALTIME_URL`, `REALTIME_PUBLISH_SECRET`
- **Security boundary:** `public` — no session/role check anywhere in this file
- **Upstash keys:** `rl:order-status:ip:{ip}` (60/60s); lock `lock:order-status:{orderId}` (5s)
- **Side effects/destructive:** `autoCleanup` hard-deletes orders older than configurable thresholds (default 12h confirmed / 10min rejected) with **no auth check on who can trigger it or what thresholds they pass**
- **Duplicate impl:** `vite.config.js` (270-338) and `server.js` (670-752) reimplement order create/list/status independently, without realtime publish in the dev-only `vite.config.js` path (needs confirmation but not found in the vite audit — a likely dev/prod behavioral gap)

3.10 `api/bookings.js`

- **Public paths / rewrites:** `/api/bookings/:restaurantId` → GET list, `/api/bookings/:id/status` → PATCH

- **Methods/actions:** `GET` (no action, `restaurantId` list); `POST` **no action** = create booking; `PATCH` or `POST ?action=updateStatus`
- **Frontend callers:** `src/lib/db.js` — `getBookings`, `createBooking`, `updateBookingStatus`
- **DB tables:** `bookings` (via `src/db/neon-bookings.js`)
- **Shared helpers/services:** Upstash rate-limit/lock (no realtime publish — bookings are not realtime-broadcast, an intentional asymmetry vs. orders)
- **Security boundary:** `public` — no session/role check
- **Upstash keys:** `rl:booking-status:ip:{ip}` (30/60s), `rl:booking-create:ip:{ip}` (5/60s); locks `lock:booking-status:{id}` (5s), `lock:booking-new:{restaurant_id}:{ip}` (300s, duplicate-submission guard)
- **Side effects:** none destructive (no delete/cleanup action exists for bookings, asymmetric with orders' `autoCleanup`)
- **Duplicate impl:** `vite.config.js` (341-372) and `server.js` (792-847)

3.11 `api/notifications.js`

- **Public paths:** `/api/notifications` (direct, no rewrites in `vercel.json` — only reachable via the catch-all `/api/(.*) → /api/$1`)
- **Methods/actions:** 14 actions total — `sendMessage`, `getMessages`, `getActiveNotification`, `publishActiveNotification`, `confirmActiveNotification`, `insertNotificationHistory`, `getNotificationHistory`, `getLatestSms`, `upsertSms`, and others per the API audit
- **Frontend callers:** `src/lib/db.js` — `sendMessage`, `getMessagesForRole`, `fetchActiveNotification`, `publishActiveNotification`, `confirmActiveNotification`, `insertNotificationHistory`, `fetchNotificationHistory`, `getLatestSmsNotification`, `upsertSmsNotification`
- **DB tables:** `messages`, `active_notification`, `notification_history`, `sms_notifications`, `help_notifications` (all via `src/db/neon-globals.js`) — **none of these five tables exist in `src/db/schema.ts`** (see §5, schema drift)
- **Security boundary:** `public` — no session/role check found

- **Side effects/destructive:** `deleteHelpNotificationNeon(id)` hard-deletes with no auth
- **Duplicate impl:** none found in `vite.config.js` / `server.js` audits — appears to be Vercel/production-only functionality (dev/prod parity gap, opposite direction of the others)

3.12 `api/system.js`

- **Public paths / rewrites:** `/api/preview-login`, `/api/preview-verify`, `/api/restaurant-db/create`, `/api/restaurant-db/drop`, `/api/restaurant-db/list`
- **Methods/actions:** `POST previewLogin`, `GET previewVerify`, `POST createRestaurantDb`, `POST dropRestaurantDb`, `GET listRestaurantDb`
- **Frontend callers:** preview-mode login gate (Replit preview auth flow, `IS_PREVIEW` mode), restaurant creation/deletion flow (indirectly, via `src/lib/db.js` `createRestaurant` / `permanentDeleteRestaurant` which also call `/api/restaurant-db/create` and `/api/restaurant-db/drop`)
- **DB tables:** `public.restaurant_databases`, and dynamically-created per-restaurant schemas `r_{shortId}` containing `orders`, `bookings`, `menu_categories`, `menu_items`
- **Shared helpers:** `bcrypt` (password hash compare), `crypto.createHmac` (token signing) — both self-contained, not shared with Better Auth
- **Env vars:** `PREVIEW_EMAIL`, `PREVIEW_PASSWORD_HASH`, `PREVIEW_SECRET`, `REPL_ID` (HMAC secret fallback), `DATABASE_URL`
- **Security boundary:**
 - `previewLogin` / `previewVerify` — `public` by design (this is the pre-auth gate for the Replit preview domain; guarded by a password + HMAC token, not Better Auth)
 - `createRestaurantDb` / `dropRestaurantDb` / `listRestaurantDb` — **public, no server-side session/role check** despite `dropRestaurantDb` performing a `DROP SCHEMA ... CASCADE` (`api/system.js:195`) — a destructive, unauthenticated operation
- **Side effects/destructive:** `dropRestaurantDb` — irreversible schema drop; graceful no-op when `DATABASE_URL` unset (i.e.,

silently does nothing on Vercel where this DB doesn't exist — see §5, dev/prod behavior difference)

- **Duplicate impl:** `vite.config.js` `preview-auth` block (20-98) reimplements the HMAC login/verify logic independently from `api/system.js`; `server.js:268-461` reimplements the restaurant-db provisioning logic independently

4. Frontend Dependency Map

All frontend access to the backend is centralized through `src/lib/db.js` (48 exported functions, each mapping to exactly one API path/action — no orphaned frontend logic bypassing this layer was found for the 12 core resources). Representative mappings, with call sites:

Resource	db.js function	Backend path	Sample caller
Restaurants	<code>getRestaurantBySlug</code>	<code>GET /api/neon/restaurant/by-slug/:slug</code>	<code>App.jsx:9,17</code> , <code>FoodDetail.jsx:13</code> , <code>RestaurantDashboard.jsx:20</code> , <code>RestaurantWebsite.jsx:36,60</code>
Orders	<code>createOrder</code>	<code>POST /api/orders</code>	<code>AddOrdersPanel.jsx:122</code> , <code>RestaurantWebsite.jsx:226</code>
Bookings	<code>createBooking</code>	<code>POST /api/bookings</code>	<code>AddBookingPanel.jsx:36</code> , <code>RestaurantWebsite.jsx:214</code>
Menu	<code>updateMenuItem</code> / <code>toggleMenuItemPublish</code>	<code>POST /api/menu/item-patch</code>	menu editor panels
Media	<code>uploadDataUrlToStorage</code>	<code>POST /api/media?action=...</code>	menu/about/logo/carousel upload UI
Settings	<code>getUserSettings</code> / <code>saveUserSettings</code>	<code>GET/POST /api/settings</code>	user preferences UI
Team	<code>getTeamMembers</code> / <code>createTeamMember</code> / <code>deleteTeamMember</code>	<code>GET/POST /api/team-members/...</code>	staff management panel
Notifications	<code>sendMessage</code> , <code>fetchActiveNotification</code> , etc.	<code>GET/POST /api/notifications</code>	notification bell / broadcast UI

Dead / broken call found: `src/lib/db.js` exports `getOrderCountThisMonth` (L492) and `getRestaurantsCreatedThisMonth` (L499), both calling `GET /api/analytics?action=...`. **There is no `api/analytics.js` file and no `vercel.json` rewrite for `/api/analytics`**. This call 404s in every environment (Vercel and the catch-all `/api/(.*) → /api/$1` in `vercel.json` will simply route it to a nonexistent file). This matches the finding from the prior serverless-function-analysis report — it has not been fixed.

No frontend caller found for: `api/menu.js` 's `getCategories` /etc. are all called; no orphaned `api/*.js` actions were identified beyond the ones noted as dead/aliased in §5.

5. Security-Boundary Summary

Function	Public write w/o auth?	Notes
<code>api/restaurants.js</code>	Yes — <code>permanentDelete</code> , <code>softDelete</code> , <code>update</code> , <code>updateProfile</code> , <code>create</code>	<code>permanentDelete</code> cascades across 7 tables with zero check
<code>api/team.js</code>	Yes — all of <code>create/update/delete</code>	Staff roster fully public-write
<code>api/menu.js</code>	Yes — all writes	Rate-limited only
<code>api/content.js</code>	Yes — all writes	Rate-limited only
<code>api/media.js</code>	Yes — all uploads	Rate-limited only; unbounded storage growth
<code>api/orders.js</code>	Yes — <code>create</code> , <code>updateStatus</code> , <code>autoCleanup</code>	<code>autoCleanup</code> deletion thresholds attacker-controlled
<code>api/bookings.js</code>	Yes — <code>create</code> , <code>updateStatus</code>	Duplicate-lock is the only safeguard
<code>api/notifications.js</code>	Yes — <code>sendMessage</code> and others	
<code>api/system.js</code>	Yes — <code>createRestaurantDb</code> , <code>dropRestaurantDb</code>	<code>dropRestaurantDb</code> is a destructive schema drop
<code>api/settings.js</code>	Partial — <code>getGlobal</code> / <code>setGlobal</code> / restaurant settings public; user settings authenticated	
<code>api/auth-check.js</code>	No — this is the check itself	Session + role verified server-side via Better Auth + <code>restaurant_members</code>
<code>api/auth.js</code>	No — this is Better Auth itself	

Root cause: `api/_lib/authz.js` provides solid primitives (`getSessionEmail` , `checkSuperadmin` , `checkRestaurantAccess` , plus Express middleware `requireSession` / `requireRestaurantAccess`), but **only** `api/auth-check.js` and select actions in `api/restaurants.js` / `api/settings.js` actually call them. The other 9 functions never import `api/_lib/authz.js` at all. This is a pre-existing, project-wide gap — not something the proposed mergers would create or fix, and per the governing rules it is reported without a remediation plan here.

Also noted: `DISABLE_AUTH=true` is currently active in this dev environment (confirmed via live workflow logs: `[auth] DISABLE_AUTH is active – authentication is bypassed`). Several of the checks above (`myIds` , user settings) are

unconditionally bypassed under this flag. This flag must never be true in the Vercel production environment; the code has no runtime safeguard against that beyond the flag itself.

6. Duplicate Implementation Summary

Every one of the 6 CRUD-heavy resources (system/preview-auth, restaurants, menu, orders, bookings, team, about/content) has its logic implemented **independently three times**:

Resource	<code>vite.config.js</code> (dev)	<code>server.js</code> (Replit runtime)	<code>api/*.js</code> (Vercel prod)
Preview auth	20-98	268-461 (via <code>_isPrivateAdminPath</code>)	<code>api/system.js</code>
Menu	100-435	470-665	<code>api/menu.js</code>
Orders	270-338	670-752	<code>api/orders.js</code>
Bookings	341-372	792-847	<code>api/bookings.js</code>
Team	(<code>menu-api</code> block)	849-887	<code>api/team.js</code>
About/Content/Media	437-564	904-1033	<code>api/content.js</code> + <code>api/media.js</code>
Restaurant DB provisioning	575-898	268-461	<code>api/system.js</code>

Confirmed behavioral drift between the three copies: -

`server.js` applies Upstash rate-limit/lock guards (e.g. `server.js:476`); the `vite.config.js` dev middleware **does not** apply the same guards, making local dev more permissive than both Replit-hosted and Vercel-hosted environments. - `server.js` has a global admin-path auth guard (`_isPrivateAdminPath` , `server.js:229`); `vite.config.js` has no equivalent, relying on per-route logic (often absent). - `api/notifications.js` exists only in the Vercel-production surface — no dev-mode (`vite.config.js`) or Replit-runtime (`server.js`) equivalent was found, meaning this feature cannot be exercised in local dev at all.

Production gaps found (routes with no reachable Vercel handler): -

`/api/migrate` — defined as a no-op in `vite.config.js:428` only; no `vercel.json` rewrite, no `api/migrate.js` . Dead in production (harmless since it's already a no-op, but misleading if someone expects it to run). - `/api/health/neon` — defined in `vite.config.js:1032` and `server.js:1114` ; no `vercel.json` rewrite. 404s on Vercel. - `PATCH /api/menu/items/:id` and `DELETE /api/menu/items/:id` — implemented in

`server.js:516,531`; `vercel.json` only rewrites `POST /api/menu/item-patch` and `POST /api/menu/item-delete`. Any client that tried the REST-ful `PATCH / DELETE` verbs directly on `/api/menu/items/:id` would 404 on Vercel even though it works against `server.js` locally. (No current frontend caller uses these verbs, so this is latent, not active, breakage.)

Duplicate aliases (within a single file, not across files): `api/team.js` treats `create`, `update`, and `shadowUpsert` as literally the same code path (`api/team.js:34`), and `delete` / `shadowDelete` as the same code path (`api/team.js:42`) — three action names, one behavior, kept for legacy frontend compatibility.

Obsolete Supabase logic: No active `@supabase/supabase-js` imports remain in `api/`, `server.js`, or `src/`. Remaining “supabase” references are confined to `attached_assets/` (historical migration notes) and standalone `supabase/*.sql` files that are not imported or executed by any current code path — dead reference material, not live code.

PART C — Merger Evaluation

7. Merger A: `settings.js` + `team.js` — DO NOT MERGE

Check	Finding
Runtime compatibility	Both plain Node/ESM Vercel functions, same <code>export default async function handler(req,res)</code> shape — compatible in isolation
HTTP method / action collisions	Blocking. <code>settings.js</code> unconditionally does <code>if (!action) return res.status(400)...</code> (<code>api/settings.js:26</code>). <code>team.js</code> ’s primary route sends no action at all — <code>/api/team-members/:restaurantId</code> → <code>/api/team?restaurantId=:restaurantId</code> (<code>vercel.json:40</code>). A merged handler using <code>settings.js</code> ’s dispatch shape would 400 every team-member list call unless the routing logic is rewritten specifically to special-case action-less requests.
<code>vercel.json</code> rewrite collisions	No direct collision in path strings, but merging requires 13 existing rewrites (6 for settings, 7 for team) to all point at one new file — an unavoidable <code>vercel.json</code> rewrite, forbidden from silently changing but technically achievable if destinations are updated in lockstep
Request parsing compatibility	Both use default Vercel JSON body parsing — compatible

Check	Finding
Response-format compatibility	Both return raw JSON values or <code>{ success: true }</code> shapes — compatible
CORS compatibility	Both call the same <code>setCors()</code> from <code>api/_lib/cors.js</code> — compatible
Auth/permission compatibility	Mismatched. <code>settings.js</code> checks session for 2 of 6 actions; <code>team.js</code> checks session for 0 of 5 actions. A merge would either import an auth requirement into <code>team.js</code> (a behavior change, forbidden without confirming callers) or leave the inconsistency in place (safe but pointless)
DB transaction separation	No shared tables (<code>user_settings</code> / <code>global_settings</code> / <code>restaurant_settings</code> vs. <code>restaurant_members</code>) — no transactional conflict
Upstash key collision risk	None — neither file uses Upstash today
Realtime event collision risk	None — neither file publishes realtime events
Payload/memory/timeout	Both trivial CRUD, no meaningful difference
Bundle/dependency risk	<code>settings.js</code> imports <code>neon()</code> from <code>src/db/pg-sql.js</code> directly; <code>team.js</code> does not — negligible size delta
Frontend callers unchanged?	Achievable only if <code>vercel.json</code> rewrites are updated to point both sets of paths at the new merged file — the URLs themselves can stay the same, but this is a config change, not a no-op
Thin-router-with-service-modules feasible?	Technically yes, but only by writing a new dispatch layer that inspects the URL/path, not just <code>?action=</code> , since <code>team.js</code> 's action-less GET has no natural place in <code>settings.js</code> 's router pattern

Verdict: DO NOT MERGE. The two files encode two different API conventions (mandatory-action vs. optional-action-with-implicit-GET). Forcing them into one handler either breaks the team-list endpoint or requires a bespoke routing rewrite that is materially more invasive than “merge two files” — the risk is disproportionate to the benefit (one fewer function, out of 12).

8. Merger B: `menu.js` + `content.js` — SAFE WITH CONDITIONS

Check	Finding
Runtime compatibility	Identical handler shape, identical import style
HTTP method / action collisions	None. <code>menu.js</code> actions: <code>getCategories</code> , <code>getItems</code> , <code>getPublishedItems</code> , <code>createItem</code> , <code>upsertItems</code> , <code>updateItem</code> , <code>deleteItem</code> , <code>upsertCategory</code> , <code>deleteCategory</code> . <code>content.js</code> actions: <code>getAbout</code> , <code>saveAbout</code> , <code>updateSocial</code> . Zero overlapping action names; both consistently require <code>?action=</code> with the same <code>400</code> fallback shape
<code>vercel.json</code> rewrite collisions	None — the 9 menu rewrites and 3 content rewrites can point at the same merged file with no path-string conflicts; ordering is irrelevant since Vercel rewrites match by source path, not destination
Request parsing	Both use default JSON body parsing — compatible

Check	Finding
Response format	Both return raw JSON — compatible
CORS	Both call the same <code>setCors()</code> — identical
Auth/permission boundary	Both are uniformly public today (no session checks in either file) — merging does not change or blend any existing permission model
DB transaction separation	Disjoint tables: <code>menu_categories</code> / <code>menu_items</code> vs. <code>restaurant_about</code> / <code>restaurants.social_links</code> — no cross-contamination risk
Upstash key collision risk	None. Menu keys are namespaced <code>rl:menu-*</code> / <code>rl:category-*</code> / <code>lock:menu-*</code> ; content keys are <code>rl:about-save</code> / <code>rl:social-update</code> . No overlapping key templates
Realtime event collision risk	None — neither publishes realtime events
Payload/memory	<code>content.js</code> 's <code>saveAbout</code> payload includes up to 4 image URLs (strings, not binary — uploads go through <code>api/media.js</code> separately); no meaningful size difference from menu payloads
Bundle/dependency risk	Combined imports: <code>src/db/neon-menu-categories.js</code> , <code>src/db/neon-menu-items.js</code> , <code>src/db/neon-restaurant-about.js</code> , <code>src/db/neon-restaurants.js</code> , plus shared <code>upstash.server.js</code> — all lightweight, no native/binary deps, no meaningful cold-start size increase
Timeout risk	Both are simple CRUD with no long-running operations — no change
Frontend callers unchanged?	Yes — every existing path stays the same string; only the <code>vercel.json</code> destination file name changes, which is invisible to the frontend
Thin-router feasible?	Yes — a merged <code>api/menu-content.js</code> can dispatch on <code>action</code> to two separate, already-existing service modules (<code>services/menu.js</code> , <code>services/content.js</code> extracted from the current inline logic) without touching either module's internals

Verdict: SAFE WITH CONDITIONS. Exact safeguards

required before merging: 1. Keep `getCategories` / `getItems` / `getPublishedItems` / `getAbout` as GET-only and everything else POST-only, exactly as today — do not relax method checks during the merge. 2. Update all 12 `vercel.json` rewrites (9 menu + 3 content) to point at the new merged filename in the same commit as the file merge — a partial update would break either menu or content in production. 3. Preserve the two independent Upstash key namespaces verbatim (`rl:menu-*` / `lock:menu-*` vs. `rl:about-*` / `rl:social-*`) — do not consolidate them into one shared key scheme, since they have different intended rate budgets. 4. Extract the current inline logic into two service modules (`menuService`, `contentService`) so the merged file is a router only, per the requirement that merges must not become a single

monolithic handler. 5. Do not use this merge as an opportunity to add authentication — that is a separate, product-approved workstream (see §5), and mixing it into this change would violate the “don’t combine security remediation with the merge implementation” rule.

9. Merger C: `orders.js` + `bookings.js` — DO NOT MERGE

Check	Finding
Runtime compatibility	Identical handler shape
HTTP method / action collisions	Blocking, twice over. (1) Both files’ primary GET route is action-less: <code>?restaurantId=X</code> → list (orders.js:17-19, bookings.js:23-25). A merged handler receiving <code>GET /api/X?restaurantId=abc</code> cannot tell whether the caller wants orders or bookings. (2) Both files’ primary POST route is action-less: a bare <code>POST</code> body creates a new record (orders.js:33-39, bookings.js:48-55). Same ambiguity for creation.
<code>vercel.json</code> rewrite collisions	To merge safely, every orders/bookings rewrite (<code>/api/orders/:restaurantId</code> , <code>/api/bookings/:restaurantId</code> , and the bare <code>POST</code> endpoints) would need a new discriminator query param (e.g. <code>&resource=orders</code>) added to the rewrite destination — a change to the routing contract, not just a file merge
Request parsing	Compatible in isolation (both default JSON body)
Response-format compatibility	Orders GET returns a plain array with <code>200</code> ; POST-create returns the created object with <code>201</code> . Bookings GET returns a plain array; POST-create returns <code>row ?? payload</code> with <code>201</code> . Similar shapes, but the ambiguity above means format compatibility is moot until routing is fixed
CORS	Both use the same <code>setCors()</code> — compatible
Auth/permission boundary	Both uniformly <code>public</code> — no blending risk here, but doesn’t offset the routing problem
DB transaction separation	Disjoint tables (<code>orders</code> vs. <code>bookings</code>) — no data risk, but irrelevant given the routing collision
Upstash key collision risk	Low in isolation — key namespaces are already resource-prefixed (<code>rl:order-*</code> / <code>lock:order-*</code> vs. <code>rl:booking-*</code> / <code>lock:booking-*</code>) — but a shared handler could accidentally cross-wire these if the dispatch logic is written carelessly, since both use nearly identical variable names (<code>ip</code> , <code>lockKey</code> , <code>allowed</code>) in structurally parallel code
Realtime event collision risk	Asymmetric, must stay isolated. <code>orders.js</code> publishes <code>ORDER_CREATED</code> / <code>ORDER_STATUS_CHANGED</code> to the Cloudflare realtime worker; <code>bookings.js</code> publishes nothing. A careless merge could accidentally wire booking creation into the same realtime publish call, silently changing product behavior (bookings would start appearing in the live order feed). Per the governing rules, realtime dispatch must not be merged unless the code

Check	Finding
	proves it's safe — here it explicitly is not, since the two flows are intentionally asymmetric
Payload-size/memory	Comparable, both small JSON payloads
Bundle/dependency risk	Combined imports add <code>src/lib/realtime-publisher.js</code> into what is currently a booking-only bundle — small but unnecessary size increase for a code path (bookings) that never uses it
Timeout risk	No change — both are fast DB operations
Frontend callers unchanged?	Only if <code>vercel.json</code> rewrites are restructured to add a resource discriminator to every affected rewrite — this is a larger routing change than the merge itself, increasing rather than decreasing overall risk
Thin-router feasible?	Feasible in principle (separate <code>ordersService</code> / <code>bookingsService</code> modules), but the routing collision means the “router” logic would have to parse the discriminator param on every single request — added complexity for a merge that saves exactly one function

Verdict: DO NOT MERGE. Both the GET-list and POST-create primary routes are structurally ambiguous without a public-contract change, and orders carry a realtime side effect that bookings must never inherit. The consolidation would require rewriting the URL contract (against the explicit instruction to preserve it) for a saving of one function out of twelve — not a favorable trade.

PART D — Recommended Architecture

10. Recommended Final Physical Function Count

11 functions (12 → 11), reflecting **only** Merger B:

`auth.js`, `auth-check.js`, `restaurants.js`, `settings.js`, `team.js`, `menu-content.js` (replaces `menu.js` + `content.js`), `media.js`, `orders.js`, `bookings.js`, `notifications.js`, `system.js`

Mergers A and C are not recommended under the current URL scheme; their potential 2-function reduction (12 → 9 combined with B) is not worth the routing-contract changes and realtime-isolation risk identified in §7 and §9.

11. Old-to-New Function Mapping

Old file	New file	Frontend-visible change
<code>api/menu.js</code>	<code>api/menu-content.js</code> (action dispatch: menu actions unchanged)	None
<code>api/content.js</code>	<code>api/menu-content.js</code> (action dispatch: content actions unchanged)	None
<code>api/settings.js</code>	(no change)	None
<code>api/team.js</code>	(no change)	None
<code>api/orders.js</code>	(no change)	None
<code>api/bookings.js</code>	(no change)	None
all others	(no change)	None

12. Public Route Compatibility Plan

For Merger B only: 1. Every existing public URL (`/api/menu/...` , `/api/about/...` , `/api/restaurant/update-social`) stays byte-for-byte identical. 2. Only the **destination** side of the 12 affected `vercel.json` rewrites changes, from `/api/menu?action=...` / `/api/content?action=...` to `/api/menu-content?action=...` . 3. No frontend file (`src/lib/db.js` or any component) requires any change, since they call the public URLs, not the internal file names. 4. Deploy the renamed/merged file and the updated `vercel.json` **in the same deploy** — never split across two deploys, since a stale rewrite pointing at a now-deleted `api/menu.js` would 404 immediately.

13. Shared Modules That Should Be Extracted

Independent of any merger decision, the real duplication problem is the `vite.config.js/server.js/api` split (§6). Recommended extractions (not yet implemented — analysis only): - `src/services/menuService.js` , `src/services/contentService.js` — pure business logic, importable by `api/menu-content.js` , `server.js` , and `vite.config.js` alike - `src/services/ordersService.js` , `src/services/bookingsService.js` — same pattern, would also let `vite.config.js` pick up the Upstash guards it currently lacks - `src/services/previewAuthService.js` — shared by `api/system.js` , `vite.config.js` 's `preview-auth` block, and `server.js` 's equivalent - A single `api/analytics.js` (or removal of the dead frontend calls) to resolve the `getOrderCountThisMonth` / `getRestaurantsCreatedThisMonth` dead-endpoint finding (§4)

14. Safe Migration Order

1. Extract `menuService` / `contentService` modules first, with `api/menu.js` and `api/content.js` still separate, each importing from the new services — verify no behavior change.
2. Update `vite.config.js` and `server.js` to import the same two services, replacing their inline duplicate logic — verify dev/Replit parity, including adding the Upstash guards that `vite.config.js` currently lacks.
3. Only after step 2 is verified stable, merge `api/menu.js` + `api/content.js` into `api/menu-content.js`, updating `vercel.json` destinations in the same change.
4. Deploy to a preview environment first; confirm all 12 affected rewrite paths still resolve correctly before promoting to production.
5. Do not attempt Mergers A or C as part of this migration; revisit only if the URL contract itself is being redesigned for other reasons.

15. Test Checklist

☐

`GET /api/menu/categories/:restaurantId` returns categories (unchanged path)

☐

`GET /api/menu/items/:restaurantId` and `.../published` return items

☐

`POST /api/menu/items`, `/api/menu/items/upsert`, `/api/menu/item-patch`, `/api/menu/item-delete` all succeed

☐

`POST /api/menu/categories/upsert`, `/api/menu/categories/delete` succeed

☐

`GET /api/about/:restaurantId` returns about content

☐

`POST /api/about/save` and `/api/restaurant/update-social` succeed

☐

All Upstash rate limits still trip at the same thresholds after the merge (spot-check `rl:menu-create` and `rl:about-save`)



Locks (`lock:menu-item:*`, `lock:menu-category:*`) still prevent concurrent deletes



CORS headers present and identical on both menu and content responses post-merge



No realtime events are emitted by any menu/content action (confirm no accidental cross-wiring)



Confirm `api/orders.js` and `api/bookings.js` remain untouched and their existing test suites (if any) still pass unmodified

16. Rollback Plan

Since Merger B is a pure file consolidation with no data migration:

1. Keep `api/menu.js` and `api/content.js` in git history (do not delete outright until the merged version has run in production without incident). 2. Rollback = revert the merge commit, which restores both original files and the original `vercel.json` rewrite destinations in one atomic change. 3. No database rollback is needed — no schema or data changes are part of this merger. 4. Because both original handlers were stateless (no in-memory state carried between requests), a rollback mid-deployment carries no risk of inconsistent state.

17. Final Go/No-Go Recommendation

- **Merger A (settings + team): NO-GO.** Blocked by incompatible dispatch conventions (§7); would require an unrelated, higher-risk `team.js` routing rewrite to proceed.
- **Merger B (menu + content): GO, with the 5 safeguards listed in §8.** Zero action collisions, zero Upstash/realtime collisions, zero auth-boundary blending, and the public API surface is fully preservable.
- **Merger C (orders + bookings): NO-GO.** Blocked by two independent routing ambiguities (GET-list and POST-create) plus

a realtime-event isolation requirement that the code does not currently prove safe to cross (§9).

- **Security gap (§5): flagged, not remediated here per the governing rules.** Recommend it be scoped as its own product-approved workstream — 9 of 12 functions currently have public write/delete access gated only by knowledge of a UUID and/or a rate limit.
- **Dead endpoint (`/api/analytics` , §4) and schema drift (5 tables used but absent from `schema.ts` , per the Drizzle audit): both should be resolved as small, independent fixes, not folded into the merge work.**